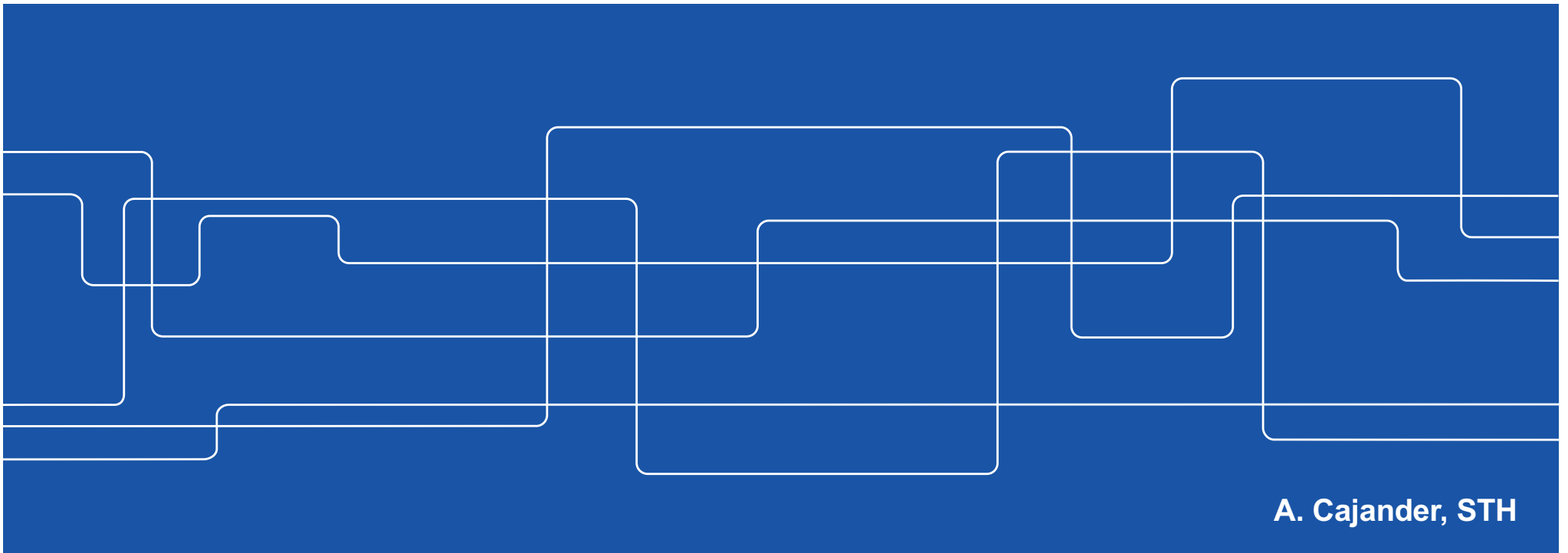




HE1025 Operativsystem

F5: Parallelism - Locks



A. Cajander, STH



Kursen...

F1: V#1 Processorn.

Ö1: V#2 c Pekare...

F2: V#3 Schemaläggning

L1: V#4 FreeRTOS

F3: V#5 Segmentation

Ö2: V#6 Buffer

F4: V#7 Paging

L2: V#8 Paging

F5: C#1 Locks & CV

Ö3: C#2 LINUX

F6: C#3 Semaphores

L3: C#4 Barberare

F7: P#1 Storage

Ö4: P#2 Buffer

F8: P#3 Communication

L4: P#4 Client/Server

Kap (1) 2..4 & 6

Kap 14 & c-boken

Kap 7..9,(10-11), RTOS

Lab #1 Handledning

Kap 12..13, 15..17

t.b.d

Kap 18-22, (23-24)

Lab #2 Handledning

Kap (25) 26..27 & 28..29.1

Kap 5 (39)

Kap 30..33, (34)

Lab #3 Handledning

Kap (35,) 36, 40, 42 (37..39, 41 & 43..46)

t.b.d

Kap (47, 39) & 48, (49..51)

Lab #4 Handledning

TEN1: 4+4p/4:e-del, minst 4p per 4:e-del för E!

LAB1: Närvaro L1..4



Inter process communication/Concurrency

Halva kursen...

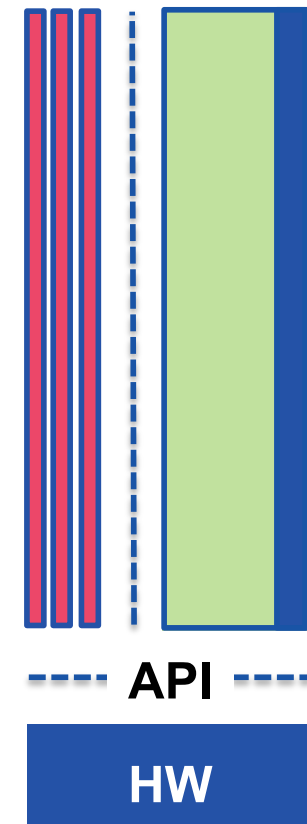
...via har n st processer (program)...

...som alla tror att de kör på en egen processor...

...med ett eget gigantiskt minne, wow!

...i själva verket är det os:et som ger sken av det genom att låta processerna köra en stund var men det krävs omfattande "städning" när en process slängs ut och en annan byts in...

...skulle processerna behöva utbyta information med varandra, inter process communication (ipc), sker det främst via tre metoder...





Inter process communication/Concurrency

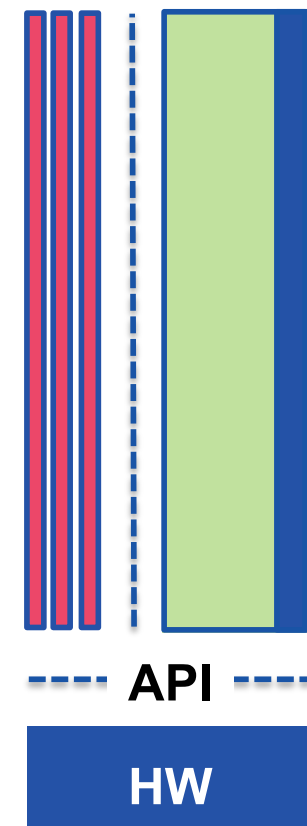
- 1) Via en fil (på något sätt)
 - 2) Via datakommunikation (på något sätt)
 - 3) Via signaler (mjukvaruavbrott) (på något sätt)
- ...mer om det i den sista ¼-delen av kursen.

Ni alla inser nog problemet...

...det känns som om det här kan gå fel...

...om inte timingen är den rätta!

Med andra ord: de första programmerarna som ställdes inför riktigt knepiga problem kopplat till parallellt exekverande kod var de som utvecklade operativsystem, därför är det än i dag ofta en del av utbildningen i operativsystem (concurrency)!



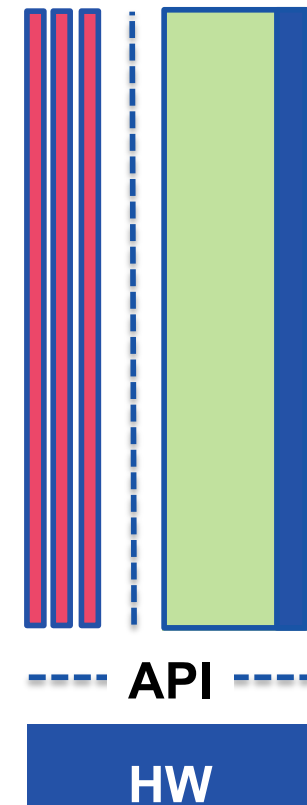


Inter process communication/Concurrency

Det har också visat sig att det är svårt att skapa operativsystem som är "rättvisa" mot I/O-intensiva processer - om en process blir "blocked", och slängs ut, så fort den gör någon form av I/O.

Att blockera den är helt enkelt ett för trubbigt instrument, processen skulle kanske ha kunnat göra något annat, medan den väntade på I/O?!

Det hela ställdes på sin spets när dagens processorer började få flera kärnor. Ska inte en process kunna dra nytta av det på något sätt?



Ett program (process) borde kunna innehålla parallella aktiviteter...

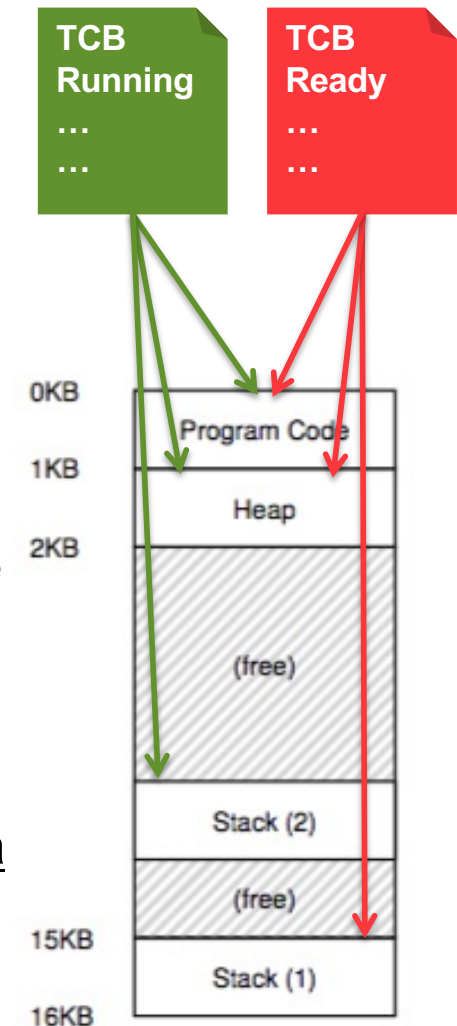


IPC/Concurrency

Insikten ledde till att konceptet trådar (threads) utvecklades. Varje tråd i en process har en egen programräknare och egna processorregister. Processens samtliga trådar delar på processens virtuella adressrymd, och har samma heap men varje tråd har sin egen stack* (men i det delade minnet!). En ny tråd ärver också alla filref. som hör till proc.

En process kan nu utvecklas så att den kan lösa flera uppgifter parallellt. Finns flera kärnor kan det vara reellt parallella i annat fall skenbart parallella. Skulle en tråd bli blockerad pga av I/O så behöver inte hela processen bli blockerad utan andra trådar kan exekvera vidare! (DB/WEB)

(* Thread local storage, reentrant, no need to change VM mapping!)



I/O-intensiva processer blir inte längre lika hårt straffade...
...och de får köra tills OS:ets policy säger att det är någon annans tur!



POSIX Threads, pthread.h

Trådar är ytterst användbara...

...många os erbjuder trådtjänster via POSIX API:et!

Portable Operating System Interface for UNIX

Standarden (IEEE 1003.1c-1995), ca 100 tjänster i 4 grupper:

- Thread management
- Mutexes
- Condition variables
- Synchronization

Standarden (IEEE 1003.1b-1993) adderar stöd för:

- Semaphores

**Det finns POSIX bibliotek för WININTEL maskiner...
...och det är mycket vanligt i inbyggda system!**



Att arbeta med trådar...

```
int pthread_create(  
    pthread_t*      thread,          // output param.  
    const pthread_attr_t* attr,      // input param.  
    void*           (*work)(void*), // a function.  
    void*           arg);            // param to work.  
  
arg_struct_t arg_struct = { n, a, b, c }; // args to work.  
  
status = pthread_create(&thread, NULL, work, &arg_struct);
```

- The thread identifier is filled in by the call and attributes are optional.
- The created thread runs the work function and then terminates.
- Typically multiple arguments are passed in a struct as above.



Att arbeta med trådar...

- A zero return value from `pthread_create()` indicates success, while a nonzero describes an error printable with `perror`.
- The work function can return a void pointer.
- Calling `pthread_join` waits for the termination of another thread and also gives access to the returned void pointer.
- A thread can only be joined once.
- There are three ways to terminate a thread:
 - 1 Return from the work function.
 - 2 The thread can call the function `void pthread_exit(void* value)`. The Standard C Library function `void exit(int status)` should normally not be used since it terminates the entire program.
 - 3 Calling the function `int pthread_cancel(pthread_t thread)` makes a request to terminate the specified thread. See cancellation below.

OK, vi kan skapa trådar...



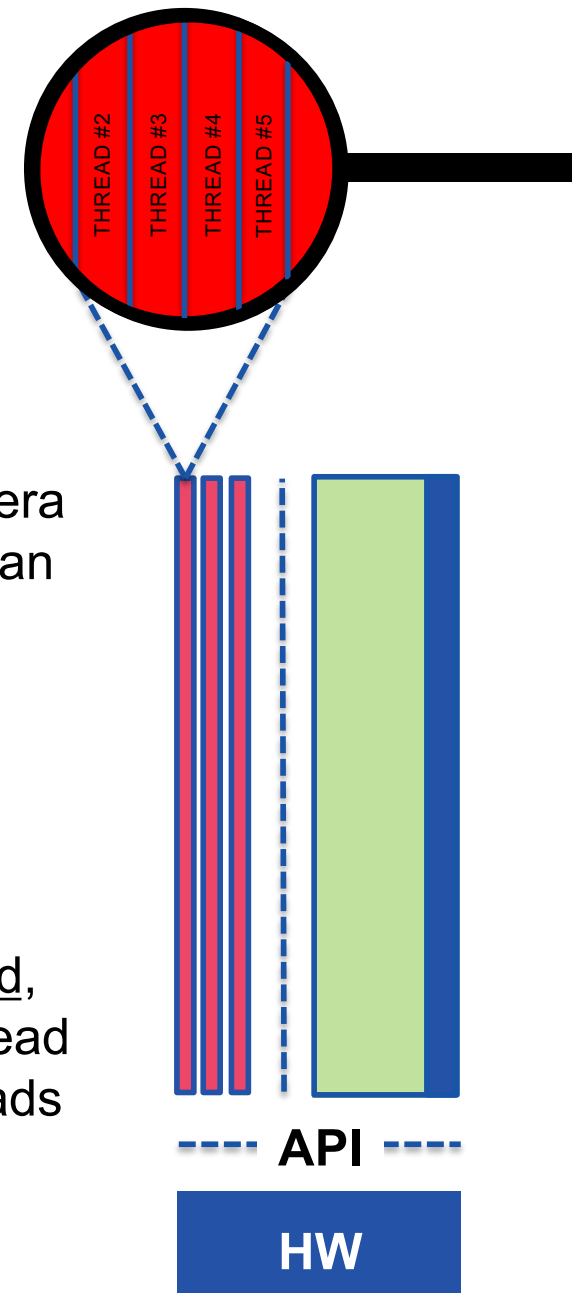
IPC/Concurrency

Om det inte redan var uppenbart...

...nu måste det finnas sätt att "säkert" kommunicera och synkronisera mellan olika trådar i annat fall kan det bli väldigt tokigt! Med andra ord det behövs mekanismer och policyer för att hantera dylikt.

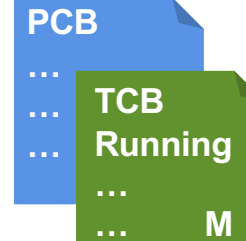
Processer med en tråd kallar vi single threaded!

Processer med flera trådar kallar vi multi threaded, och de har då uppenbart också egna interna Thread Control Blocks (TCB) som håller reda på varje tråds interna tillstånd!



Vilka problem kan då uppstå?

Ett första exempel...

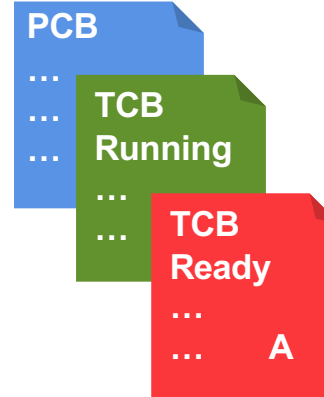


```
1  #include <stdio.h>
2  #include <assert.h>
3  #include <pthread.h>
4
5  void *mythread(void *arg) {
6      printf("%s\n", (char *) arg);
7      return NULL;
8  }
9
10 int
11 main(int argc, char *argv[]) {
12     pthread_t p1, p2;
13     int rc;
14     printf("main: begin\n");
15     rc = pthread_create(&p1, NULL, mythread, "A");
16     rc = pthread_create(&p2, NULL, mythread, "B");
17     // join waits for the threads to finish
18     rc = pthread_join(p1, NULL); assert(rc == 0);
19     rc = pthread_join(p2, NULL); assert(rc == 0);
20     printf("main: end\n");
21     return 0;
22 }
```

main: begin

Ett första exempel...

```
1  #include <stdio.h>
2  #include <assert.h>
3  #include <pthread.h>
4
5  void *mythread(void *arg) {
6      printf("%s\n", (char *) arg);
7      return NULL;
8  }
9
10 int
11 main(int argc, char *argv[]) {
12     pthread_t p1, p2;
13     int rc;
14     printf("main: begin\n");
15     rc = pthread_create(&p1, NULL, mythread, "A");
16     rc = pthread_create(&p2, NULL, mythread, "B");
17     // join waits for the threads to finish
18     rc = pthread_join(p1, NULL); assert(rc == 0);
19     rc = pthread_join(p2, NULL); assert(rc == 0);
20     printf("main: end\n");
21     return 0;
22 }
```

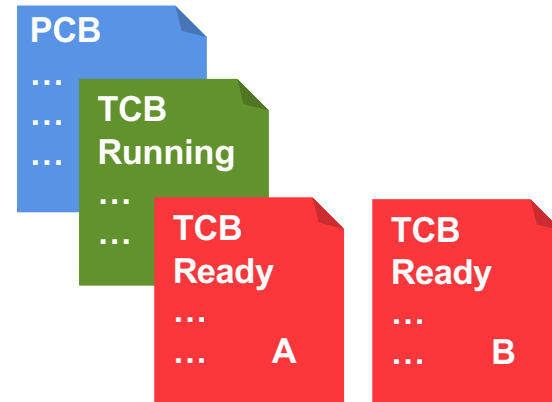


main: begin



Ett första exempel...

```
1  #include <stdio.h>
2  #include <assert.h>
3  #include <pthread.h>
4
5  void *mythread(void *arg) {
6      printf("%s\n", (char *) arg);
7      return NULL;
8  }
9
10 int
11 main(int argc, char *argv[]) {
12     pthread_t p1, p2;
13     int rc;
14     printf("main: begin\n");
15     rc = pthread_create(&p1, NULL, mythread, "A");
16     rc = pthread_create(&p2, NULL, mythread, "B");
17     // join waits for the threads to finish
18     rc = pthread_join(p1, NULL); assert(rc == 0);
19     rc = pthread_join(p2, NULL); assert(rc == 0);
20     printf("main: end\n");
21     return 0;
22 }
```

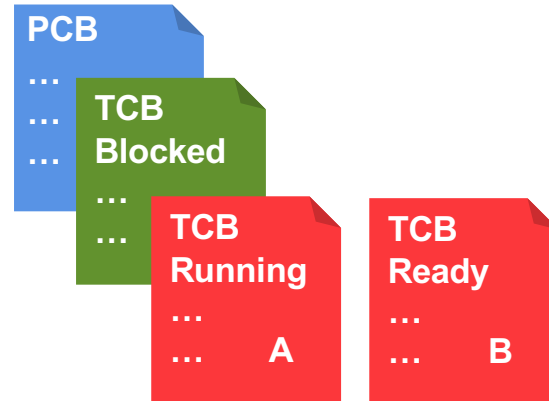


main: begin



Ett första exempel...

```
1  #include <stdio.h>
2  #include <assert.h>
3  #include <pthread.h>
4
5  void *mythread(void *arg) {
6      printf("%s\n", (char *) arg);
7      return NULL;
8  }
9
10 int
11 main(int argc, char *argv[]) {
12     pthread_t p1, p2;
13     int rc;
14     printf("main: begin\n");
15     rc = pthread_create(&p1, NULL, mythread, "A");
16     rc = pthread_create(&p2, NULL, mythread, "B");
17     // join waits for the threads to finish
18     rc = pthread_join(p1, NULL); assert(rc == 0);
19     rc = pthread_join(p2, NULL); assert(rc == 0);
20     printf("main: end\n");
21     return 0;
22 }
```



main: begin
A



Ett första exempel...

```
1  #include <stdio.h>
2  #include <assert.h>
3  #include <pthread.h>
4
5  void *mythread(void *arg) {
6      printf("%s\n", (char *) arg);
7      return NULL;
8  }
9
10 int
11 main(int argc, char *argv[]) {
12     pthread_t p1, p2;
13     int rc;
14     printf("main: begin\n");
15     rc = pthread_create(&p1, NULL, mythread, "A");
16     rc = pthread_create(&p2, NULL, mythread, "B");
17     // join waits for the threads to finish
18     rc = pthread_join(p1, NULL); assert(rc == 0);
19     rc = pthread_join(p2, NULL); assert(rc == 0);
20     printf("main: end\n");
21     return 0;
22 }
```

PCB

...
...
...

TCB
Running

...
... M

TCB
Ready

...
... B

main: begin

A



Ett första exempel...

```
1  #include <stdio.h>
2  #include <assert.h>
3  #include <pthread.h>
4
5  void *mythread(void *arg) {
6      printf("%s\n", (char *) arg);
7      return NULL;
8  }
9
10 int
11 main(int argc, char *argv[]) {
12     pthread_t p1, p2;
13     int rc;
14     printf("main: begin\n");
15     rc = pthread_create(&p1, NULL, mythread, "A");
16     rc = pthread_create(&p2, NULL, mythread, "B");
17     // join waits for the threads to finish
18     rc = pthread_join(p1, NULL); assert(rc == 0);
19     rc = pthread_join(p2, NULL); assert(rc == 0);
20     printf("main: end\n");
21     return 0;
22 }
```

PCB

...
...
...

TCB
Blocked

...
... M

TCB
Running

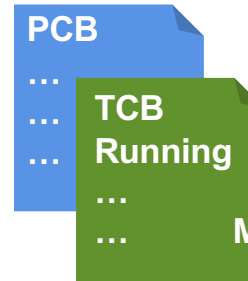
...
... B

main: begin

A
B



Ett första exempel...



```
1  #include <stdio.h>
2  #include <assert.h>
3  #include <pthread.h>
4
5  void *mythread(void *arg) {
6      printf("%s\n", (char *) arg);
7      return NULL;
8  }
9
10 int
11 main(int argc, char *argv[]) {
12     pthread_t p1, p2;
13     int rc;
14     printf("main: begin\n");
15     rc = pthread_create(&p1, NULL, mythread, "A");
16     rc = pthread_create(&p2, NULL, mythread, "B");
17     // join waits for the threads to finish
18     rc = pthread_join(p1, NULL); assert(rc == 0);
19     rc = pthread_join(p2, NULL); assert(rc == 0);
20     printf("main: end\n");
21     return 0;
22 }
```

main: begin

A

B

main: end

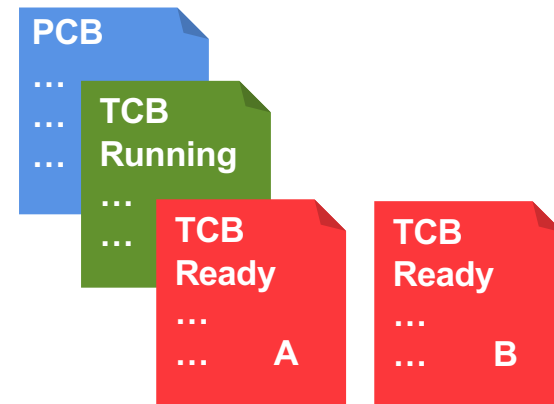
Wow!

Men var det här hela sanningen?!



Ett första exempel...

```
1  #include <stdio.h>
2  #include <assert.h>
3  #include <pthread.h>
4
5  void *mythread(void *arg) {
6      printf("%s\n", (char *) arg);
7      return NULL;
8  }
9
10 int
11 main(int argc, char *argv[]) {
12     pthread_t p1, p2;
13     int rc;
14     printf("main: begin\n");
15     rc = pthread_create(&p1, NULL, mythread, "A");
16     rc = pthread_create(&p2, NULL, mythread, "B");
17     // join waits for the threads to finish
18     rc = pthread_join(p1, NULL); assert(rc == 0);
19     rc = pthread_join(p2, NULL); assert(rc == 0);
20     printf("main: end\n");
21     return 0;
22 }
```



main: begin
B
A
main: end

Trådarna kan ha körs i en helt annan ordning... ...och det blir värre!

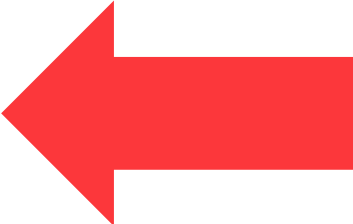


Ett andra exempel...



```
prompt> gcc -o main main.c -Wall -pthread
prompt> ./main
main: begin (counter = 0)
A: begin
B: begin
A: done
B: done
main: done with both (counter = 20000000)
```

OK?!



```
1  #include <stdio.h>
2  #include <pthread.h>
3  #include "mythreads.h"
4
5  static volatile int counter = 0;
6
7  //
8  // mythread()
9  //
10 // Simply adds 1 to counter repeatedly, in a loop
11 // No, this is not how you would add 10,000,000 to
12 // a counter, but it shows the problem nicely.
13 //
14 void *
15 mythread(void *arg)
16 {
17     printf("%s: begin\n", (char *) arg);
18     int i;
19     for (i = 0; i < 1e7; i++) {
20         counter = counter + 1;
21     }
22     printf("%s: done\n", (char *) arg);
23     return NULL;
24 }
25
26 //
27 // main()
28 //
29 // Just launches two threads (pthread_create)
30 // and then waits for them (pthread_join)
31 //
32 int
33 main(int argc, char *argv[])
34 {
35     pthread_t p1, p2;
36     printf("main: begin (counter = %d)\n", counter);
37     Pthread_create(&p1, NULL, mythread, "A");
38     Pthread_create(&p2, NULL, mythread, "B");
39
40     // join waits for the threads to finish
41     Pthread_join(p1, NULL);
42     Pthread_join(p2, NULL);
43     printf("main: done with both (counter = %d)\n", counter);
44     return 0;
45 }
```



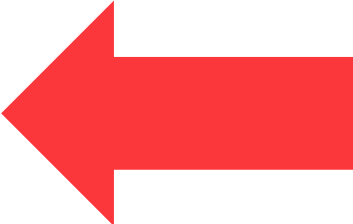
Ett andra exempel...



```
prompt> ./main
main: begin (counter = 0)
A: begin
B: begin
A: done
B: done
main: done with both (counter = 19221041)
B: done
main: done with both (counter = 19345221)
```

Svaret kan bli vad som helst...
...och det är inte förutsägbart...
...prog. är inte deterministiskt!

Vad händer?



```
1  #include <stdio.h>
2  #include <pthread.h>
3  #include "mythreads.h"
4
5  static volatile int counter = 0;
6
7  //
8  // mythread()
9  //
10 // Simply adds 1 to counter repeatedly, in a loop
11 // No, this is not how you would add 10,000,000 to
12 // a counter, but it shows the problem nicely.
13 //
14 void *
15 mythread(void *arg)
16 {
17     printf("%s: begin\n", (char *) arg);
18     int i;
19     for (i = 0; i < 1e7; i++) {
20         counter = counter + 1;
21     }
22     printf("%s: done\n", (char *) arg);
23     return NULL;
24 }
25
26 //
27 // main()
28 //
29 // Just launches two threads (pthread_create)
30 // and then waits for them (pthread_join)
31 //
32 int
33 main(int argc, char *argv[])
34 {
35     pthread_t p1, p2;
36     printf("main: begin (counter = %d)\n", counter);
37     Pthread_create(&p1, NULL, mythread, "A");
38     Pthread_create(&p2, NULL, mythread, "B");
39
40     // join waits for the threads to finish
41     Pthread_join(p1, NULL);
42     Pthread_join(p2, NULL);
43     printf("main: done with both (counter = %d)\n", counter);
44     return 0;
45 }
```




Ett andra exempel...

SXXCNT 0x000 ;counter

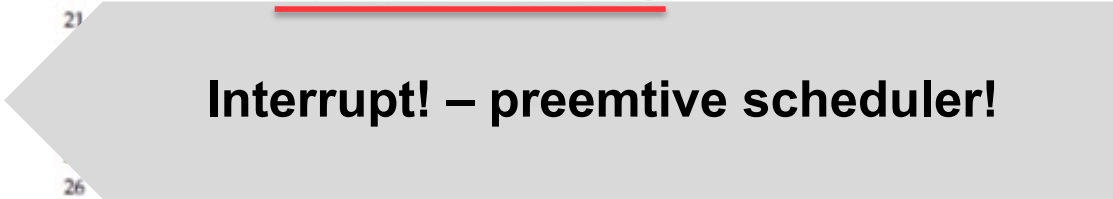
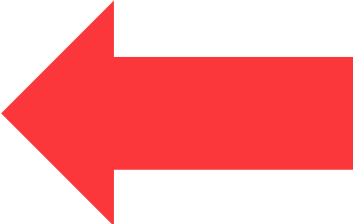
"A"

MOVF SXXCNT,W ;load cnt
ADDLW 0x01 ;+1
MOVWF SXXCNT ;save cnt

"B"

MOVF SXXCNT,W ;load cnt
ADDLW 0x01 ;+1
MOVWF SXXCNT ;save cnt

```
1  #include <stdio.h>
2  #include <pthread.h>
3  #include "mythreads.h"
4
5  static volatile int counter = 0;
6
7  //
8  // mythread()
9  //
10 // Simply adds 1 to counter repeatedly, in a loop
11 // No, this is not how you would add 10,000,000 to
12 // a counter, but it shows the problem nicely.
13 //
14 void *
15 mythread(void *arg)
16 {
17     printf("%s: begin\n", (char *) arg);
18     int i;
19     for (i = 0; i < 1e7; i++) {
20         counter = counter + 1;
21     }
22
23
24
25
26 // main()
27 //
28 // Just launches two threads (pthread_create)
29 // and then waits for them (pthread_join)
30 //
31 //
32 int
33 main(int argc, char *argv[])
34 {
35     pthread_t p1, p2;
36     printf("main: begin (counter = %d)\n", counter);
37     Pthread_create(&p1, NULL, mythread, "A");
38     Pthread_create(&p2, NULL, mythread, "B");
39
40     // join waits for the threads to finish
41     Pthread_join(p1, NULL);
42     Pthread_join(p2, NULL);
43     printf("main: done with both (counter = %d)\n", counter);
44     return 0;
45 }
```



Interrupt! – preemtive scheduler!



Ett andra exempel...

SXXCNT 0x000 ;counter

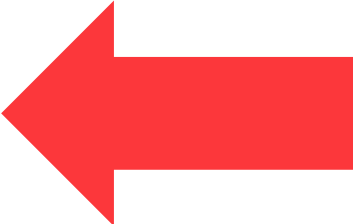
"A"

MOVF SXXCNT,W ;load cnt
ADDLW 0x01 ;+1
MOVWF SXXCNT ;save cnt

"B"

MOVF SXXCNT,W ;load cnt
ADDLW 0x01 ;+1
MOVWF SXXCNT ;save cnt

```
1  #include <stdio.h>
2  #include <pthread.h>
3  #include "mythreads.h"
4
5  static volatile int counter = 0;
6
7  //
8  // mythread()
9  //
10 // Simply adds 1 to counter repeatedly, in a loop
11 // No, this is not how you would add 10,000,000 to
12 // a counter, but it shows the problem nicely.
13 //
14 void *
15 mythread(void *arg)
16 {
17     printf("%s: begin\n", (char *) arg);
18     int i;
19     for (i = 0; i < 1e7; i++) {
20         counter = counter + 1;
21     }
22
23     // main()
24     //
25     // Just launches two threads (pthread_create)
26     // and then waits for them (pthread_join)
27     //
28
29     Pthread_create(&p2, NULL, mythread, "B");
30
31     // join waits for the threads to finish
32     Pthread_join(p1, NULL);
33     Pthread_join(p2, NULL);
34     printf("main: done with both (counter = %d)\n", counter);
35     return 0;
36 }
```



WREG = "Ögonblicksbild" av Counter (för den här tråden)
Tråden Ready (not running)

Tråd B läser av Counter..
...ökar värdet med ett..
...& skriver tillbaks det nya värdet!



Ett andra exempel...

SXXCNT 0x000 ;counter

"A"

MOVF SXXCNT,W ;load cnt
ADDLW 0x01 ;+1
MOVWF SXXCNT ;save cnt

"B"

MOVF SXXCNT,W ;load cnt
ADDLW 0x01 ;+1
MOVWF SXXCNT ;save cnt

+1, inte +2!
Shared data
Race condition

```
1  #include <stdio.h>
2  #include <pthread.h>
3  #include "mythreads.h"
4
5  static volatile int counter = 0;
6
7  //
8  // mythread()
9  //
10 // Simply adds 1 to counter repeatedly, in a loop
11 // No, this is not how you would add 10,000,000 to
12 // a counter, but it shows the problem nicely.
13 //
14 void *
15 mythread(void *arg)
16 {
17     printf("%s: begin\n", (char *) arg);
18     int i;
19     for (i = 0; i < 1e7; i++) {
20         counter = counter + 1;
21     }
22
23
24
25
26
27 // main()
28 //
29 // Just launches two threads (pthread_create)
30 // and then waits for them (pthread_join)
31 //
32 int
33 main(int argc, char *argv[])
34 {
35     pthread_t p1, p2;
36     printf("main: begin (counter = %d)\n", counter);
37     Pthread_create(&p1, NULL, mythread, "A");
38     Pthread_create(&p2, NULL, mythread, "B");
39
40     // join waits for the threads to finish
41     Pthread_join(p1, NULL);
42     Pthread_join(p2, NULL);
43     printf("main: done with both (counter = %d)\n", counter);
44     return 0;
45 }
```




Ett andra exempel...

SXXCNT 0x000 ;counter

"A"

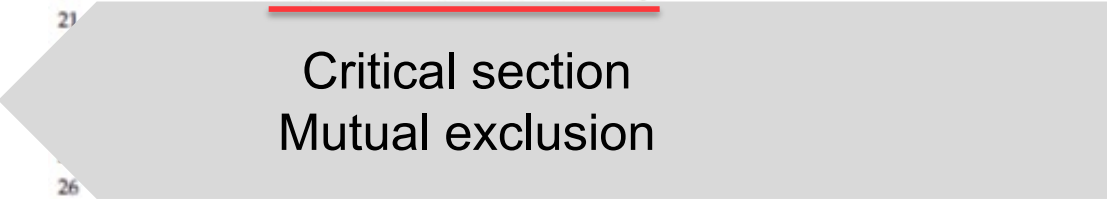
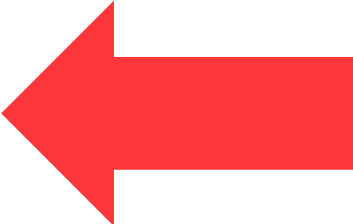
MOVF SXXCNT,W ;load cnt
ADDLW 0x01 ;+1
MOVWF SXXCNT ;save cnt

"B"

MOVF SXXCNT,W ;load cnt
ADDLW 0x01 ;+1
MOVWF SXXCNT ;save cnt

Vad hade hänt om
Kompilatorn använt
INCF SXXCNT?!

```
1  #include <stdio.h>
2  #include <pthread.h>
3  #include "mythreads.h"
4
5  static volatile int counter = 0;
6
7  //
8  // mythread()
9  //
10 // Simply adds 1 to counter repeatedly, in a loop
11 // No, this is not how you would add 10,000,000 to
12 // a counter, but it shows the problem nicely.
13 //
14 void *
15 mythread(void *arg)
16 {
17     printf("%s: begin\n", (char *) arg);
18     int i;
19     for (i = 0; i < 1e7; i++) {
20         counter = counter + 1;
21     }
22
23     // main()
24     //
25     // Just launches two threads (pthread_create)
26     // and then waits for them (pthread_join)
27     //
28     int
29     main(int argc, char *argv[])
30     {
31         pthread_t p1, p2;
32         printf("main: begin (counter = %d)\n", counter);
33         Pthread_create(&p1, NULL, mythread, "A");
34         Pthread_create(&p2, NULL, mythread, "B");
35
36         // join waits for the threads to finish
37         Pthread_join(p1, NULL);
38         Pthread_join(p2, NULL);
39         printf("main: done with both (counter = %d)\n", counter);
40         return 0;
41     }
42 }
```



Critical section
Mutual exclusion



Ett andra exempel...

SXXCNT 0x000 ;counter

"A"

INCF SXXCN ;cnt+1

"B"

INCF SXXCN ;cnt+1

INCF gör hela jobbet
i ett odelbart steg,
den är atomär!

```
1  #include <stdio.h>
2  #include <pthread.h>
3  #include "mythreads.h"
4
5  static volatile int counter = 0;
6
7  //
8  // mythread()
9  //
10 // Simply adds 1 to counter repeatedly, in a loop
11 // No, this is not how you would add 10,000,000 to
12 // a counter, but it shows the problem nicely.
13 //
14 void *
15 mythread(void *arg)
16 {
17     printf("%s: begin\n", (char *) arg);
18     int i;
19     for (i = 0; i < 1e7; i++) {
20         counter = counter + 1;
21     }
22
23     // Critical section
24     // Mutual exclusion
25
26
27 // main()
28 //
29 // Just launches two threads (pthread_create)
30 // and then waits for them (pthread_join)
31 //
32 int
33 main(int argc, char *argv[])
34 {
35     pthread_t p1, p2;
36     printf("main: begin (counter = %d)\n", counter);
37     Pthread_create(&p1, NULL, mythread, "A");
38     Pthread_create(&p2, NULL, mythread, "B");
39
40     // join waits for the threads to finish
41     Pthread_join(p1, NULL);
42     Pthread_join(p2, NULL);
43     printf("main: done with both (counter = %d)\n", counter);
44     return 0;
45 }
```



Med andra ord...

Det lär knappast finnas atomära instruktioner för allt!

1) Med andra ord så måste finnas några atomära special-instruktioner som gör det möjligt att skapa generella synkroniseringsfunktioner (lock) som kan användas för att skapa mutual exclusion för critical sectors. (Test-and-set)

2) Det finns ett till problem också som kan uppstå när en tråd måste vänta på att en annan tråd blir klar innan den får fortsätta (wait for another). För att hantera det behövs något som kallas condition variables, vi återkommer till det.

3) För att toppa detta kan det också bli riktigt lurigt om olika trådar väntar på resurser, som andra trådar håller på, så att ingen kan köra!



Cache/Total Store Order

Dagens processorer tar alltid hjälp av instruktions cache...

...dvs instruktioner hämtas i förväg (på chans)...

...och utförs ibland i förväg (på chans)...

...till och med i oordning ibland (r går före w)!

Finns det flera kärnor kan det bli väldigt tokigt...

...begreppet Total Store Order...

...är den ordning av minnesacc. som en annan kärna "ser"...

...men det behöver inte vara den faktiska ordningen!



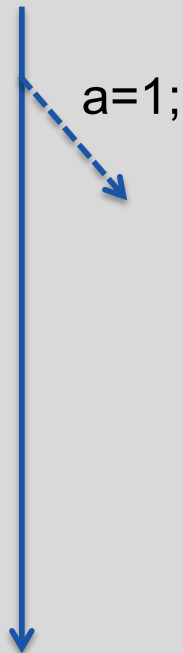
Obs följande beror inte på avbrott!

Tråd X
Core #1

int a=0;

int b=0;

Tråd Y
Core #2



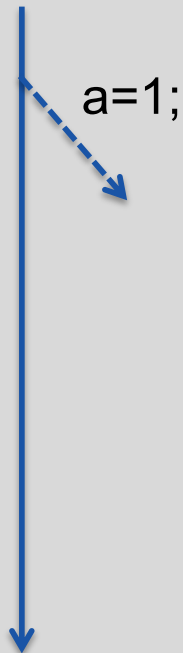
Obs följande beror inte på avbrott!

Tråd X
Core #1

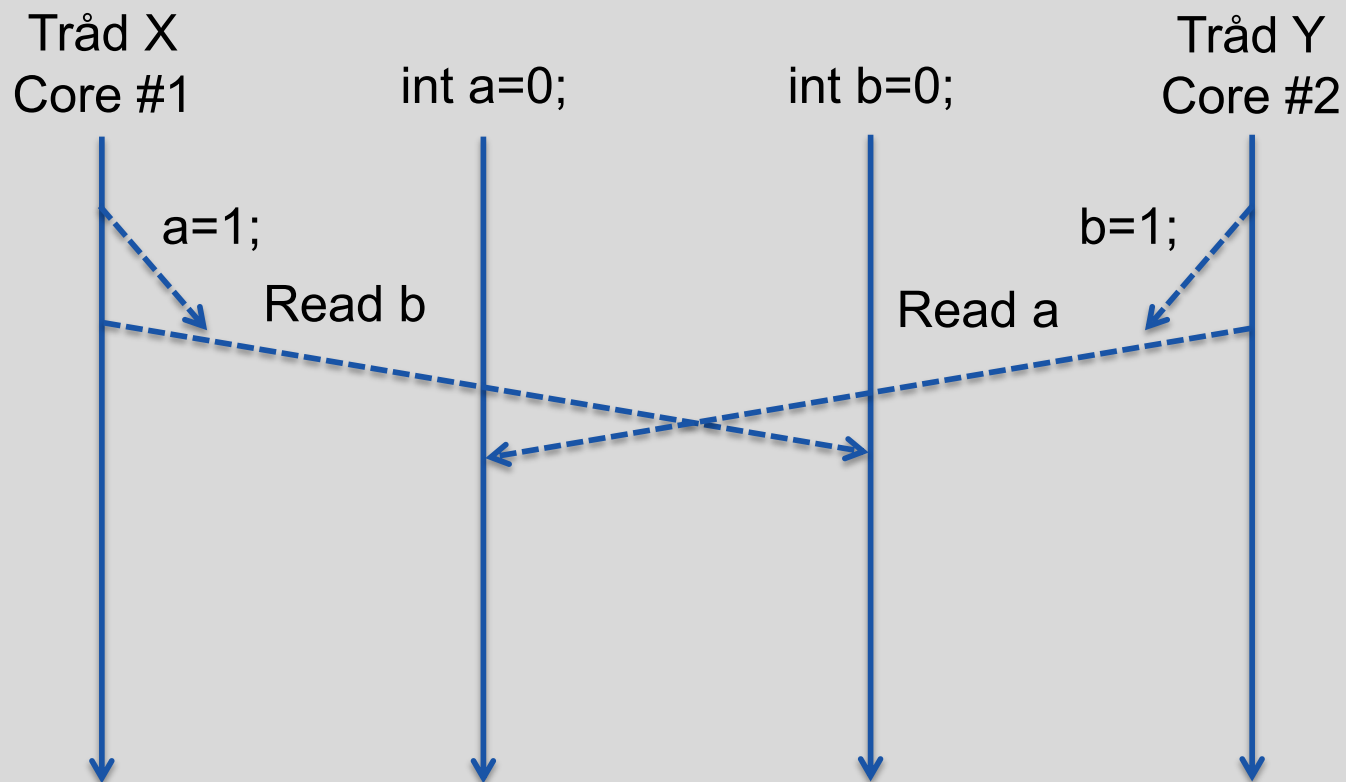
int a=0;

int b=0;

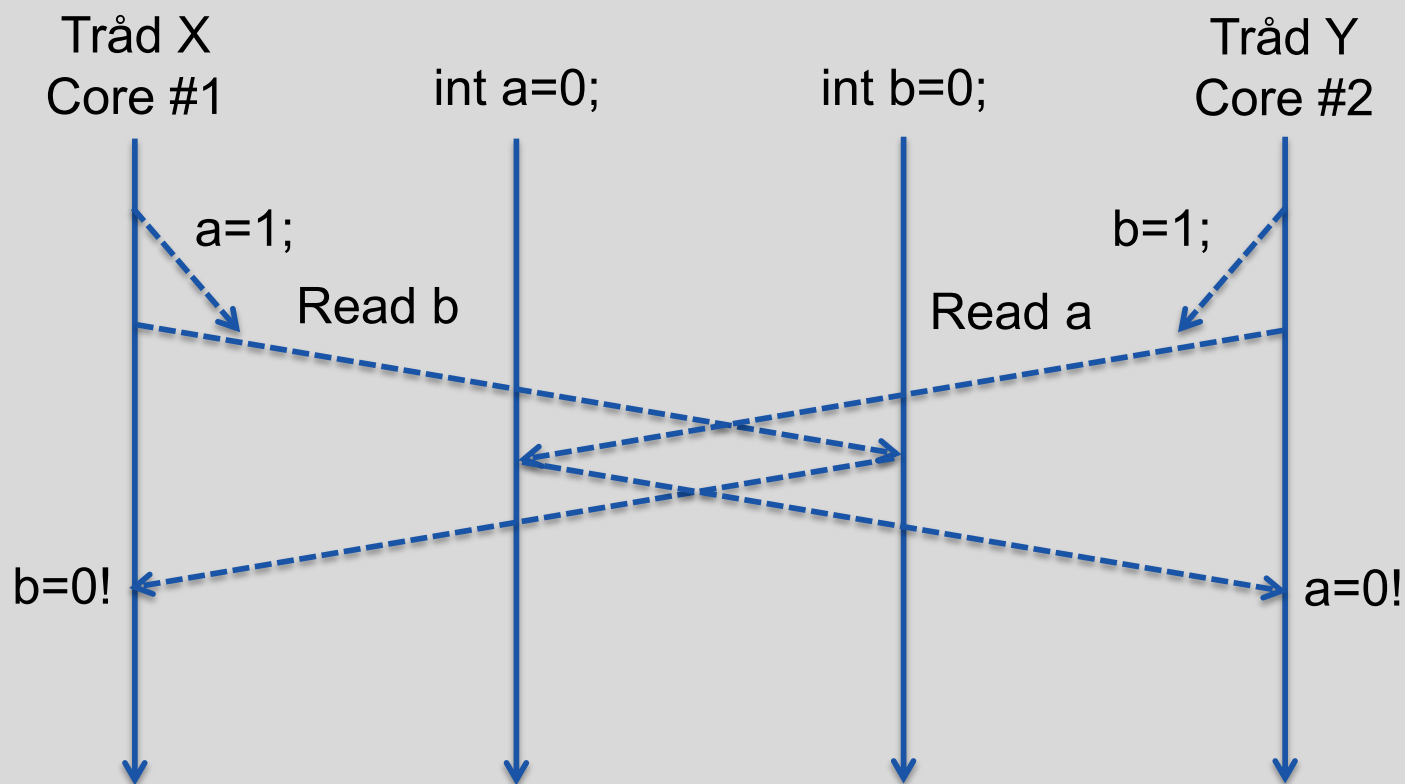
Tråd Y
Core #2



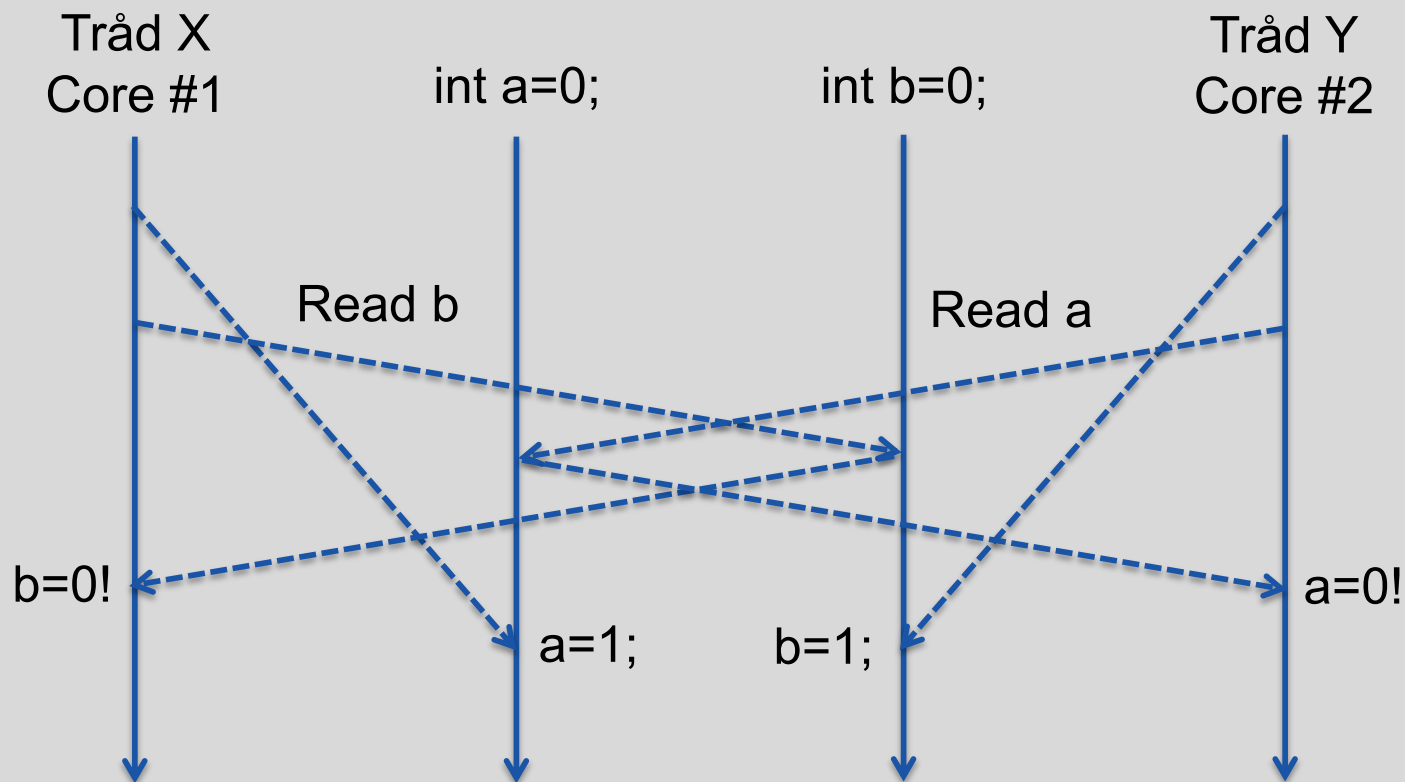
Obs följande beror inte på avbrott!



Obs följande beror inte på avbrott!



Obs följande beror inte på avbrott!





Locks, principiellt...

```
balance = balance + 1;
```

Critical section => needs mutual exclusion



```
lock_t mutex;
```

```
...
```

```
lock(&mutex);
```

```
balance = balance + 1;
```

```
unlock(&mutex);
```

En variabel som är låset, som måste initieras!

Försök att låsa! (available/free*)

(Bara en tråd kan vara här åt gången!)

Lämna den kritiska sektionen & återställ låset!

(*Motsatsen är: acquired, locked or held)

Går det inte att "låsa" -> "vänta" till det går...



Hur skapar vi funktionerna lock/unlock()?

Som vanligt måste implementeringen vara:

- Fungerande – verkligen ge skydd.
- Rättvis – så att ingen tråd försummas (fair/starve).
- Effektiv – inte förbruka processorkraft i onödan.

Att stänga av interrupt är inte en lösning*:

- Priviligierad instruktion, inte för gemene man...
- Flerkärniga processorer, stoppa alla kärnor...
- Latency, kan inte hantera nya avbrott, tappar avbrott...

(Möjligen när os:et kör egen kärnkod kan det vara ok.)

(*varför skulle det kunna vara en lösning? (Embedded))



Ett första försök...

```
typedef struct __lock_t { int flag; } lock_t;

void init(lock_t *mutex) {
    // 0 -> lock is available, 1 -> held
    mutex->flag = 0;
}

void lock(lock_t *mutex) {
    while (mutex->flag == 1) // TEST the flag
        ; // spin-wait (do nothing)
    mutex->flag = 1;         // now SET it!
}

void unlock(lock_t *mutex) {
    mutex->flag = 0;
}
```



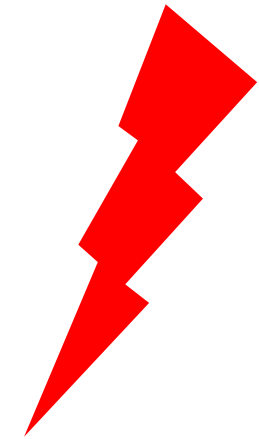

Ett första försök...

```
typedef struct __lock_t { int flag; } lock_t;

void init(lock_t *mutex) {
    // 0 -> lock is available, 1 -> held
    mutex->flag = 0;
}

void lock(lock_t *mutex) {
    while (mutex->flag == 1) // TEST the flag
        ; // spin-wait (do nothing)
    mutex->flag = 1;         // now SET it!
}

void unlock(lock_t *mutex) {
    mutex->flag = 0;
}
```





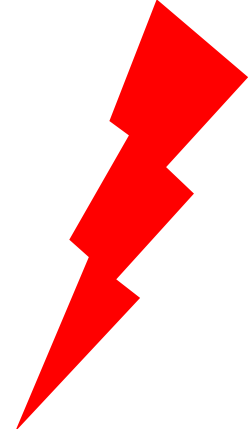
Ett första försök...

```
typedef struct __lock_t { int flag; } lock_t;

void init(lock_t *mutex) {
    // 0 -> lock is available, 1 -> held
    mutex->flag = 0;
}

void lock(lock_t *mutex) {
    while (mutex->flag == 1) // TEST the flag
        ; // spin-wait (do nothing)
    mutex->flag = 1;         // now SET it!
}

void unlock(lock_t *mutex) {
    mutex->flag = 0;
}
```

- 
- ~~Fungerande~~
 - Rättvist
 - Effektiv



Försök två...

```
int TestAndSet(int *old_ptr, int new) {  
    int old = *old_ptr; // fetch old value at old_ptr  
    *old_ptr = new;      // store 'new' into old_ptr  
    return old;          // return the old value  
}
```

```
typedef struct __lock_t {  
    int flag;  
} lock_t;  
  
void init(lock_t *lock) {  
    // 0 indicates that lock is available, 1 that it is held  
    lock->flag = 0;  
}  
  
void lock(lock_t *lock) {  
    while (TestAndSet(&lock->flag, 1) == 1)  
        ; // spin-wait (do nothing)  
}  
  
void unlock(lock_t *lock) {  
    lock->flag = 0;  
}
```

(aka Spin Lock using Test-and-set)



Försök två...

```
int TestAndSet(int *old_ptr, int new) {  
    int old = *old_ptr; // fetch old value at old_ptr  
    *old_ptr = new;      // store 'new' into old_ptr  
    return old;          // return the old value  
}
```

```
typedef struct __lock_t {  
    int flag;  
} lock_t;  
  
void init(lock_t *lock) {  
    // 0 indicates that lock is available, 1 that it is held  
    lock->flag = 0;  
}  
  
void lock(lock_t *lock) {  
    while (TestAndSet(&lock->flag, 1) == 1)  
        ; // spin-wait (do nothing)  
}  
  
void unlock(lock_t *lock) {  
    lock->flag = 0;  
}
```

- **Fungerande**
- **Rättvist**
- **Effektiv**

Finns inget stöd för att säkerställa rättvisa, men med tur...
Viss effektivitet om processorn är flerkärning...



Compare-and-swap/exchange (sparc/x86)

```
int CompareAndSwap(int *ptr, int expected, int new) {  
    int actual = *ptr;  
    if (actual == expected)  
        *ptr = new;  
    return actual;  
}
```

```
void lock(lock_t *lock) {  
    while (CompareAndSwap(&lock->flag, 0, 1) == 1)  
        ; // spin  
}
```



Load-Linked & Store-conditional (PPC,ARM)

```
int LoadLinked(int *ptr) {  
    return *ptr;  
}  
  
int StoreConditional(int *ptr, int value) {  
    if (no one has updated *ptr since the LoadLinked to this address) {  
        *ptr = value;  
        return 1; // success!  
    } else {  
        return 0; // failed to update  
    }  
}
```

```
void lock(lock_t *lock) {  
    while (1) {  
        while (LoadLinked(&lock->flag) == 1)  
            ; // spin until it's zero  
        if (StoreConditional(&lock->flag, 1) == 1)  
            return; // if set-it-to-1 was a success: all done  
                    // otherwise: try it all over again  
    }  
}
```



Fetch-and-add (ticket lock)

```
int FetchAndAdd(int *ptr) {  
    int old = *ptr;  
    *ptr = old + 1;  
    return old;  
}
```

```
typedef struct __lock_t {  
    int ticket;  
    int turn;  
} lock_t;  
  
void lock_init(lock_t *lock) {  
    lock->ticket = 0;  
    lock->turn = 0;  
}  
  
void lock(lock_t *lock) {  
    int myturn = FetchAndAdd(&lock->ticket);  
    while (lock->turn != myturn)  
        ; // spin  
}  
  
void unlock(lock_t *lock) {  
    lock->turn = lock->turn + 1;  
}
```

- **Fungerande**
- **Rättvist**
- **Effektiv**

Vi måste komma åt loop-problemet...



Lämna företräde (Yield)...

Om det fanns ett systemanrop för en running thread...
...som helt enkelt gav upp processorn (yield)...
...skulle tråden inte loopa tills tilldelad tid var slut!

```
void init() {  
    flag = 0;  
}  
  
void lock() {  
    while (TestAndSet(&flag, 1) == 1)  
        yield(); // give up the CPU  
}  
  
void unlock() {  
    flag = 0;  
}
```

(Priority Inversion problem)

- **Fungerande**
- **Rättvist**
- **Effektiv**

...men problemet med rättvisa kvarstår...



...det behövs nog någon typ av turordning!

```
typedef struct __lock_t {
    int flag;
    int guard;
    queue_t *q;
} lock_t;

void lock_init(lock_t *m) {
    m->flag = 0;
    m->guard = 0;
    queue_init(m->q);
}

void lock(lock_t *m) {
    while (TestAndSet(&m->guard, 1) == 1)
        ; //acquire guard lock by spinning
    if (m->flag == 0) {
        m->flag = 1; // lock is acquired
        m->guard = 0;
    } else {
        queue_add(m->q, gettid());
        m->guard = 0;
        park();
    }
}
```



...det behövs nog någon typ av turordning!

```
typedef struct __lock_t {
    int flag;
    int guard;
    queue_t *q;
} lock_t;

void lock_init(lock_t *m) {
    m->flag = 0;
    m->guard = 0;
    queue_init(m->q);
}

void lock(lock_t *m) {
    while (TestAndSet(&m->guard, 1) == 1)
        ; //acquire guard lock by spinning
    if (m->flag == 0) {
        m->flag = 1; // lock is acquired
        m->guard = 0;
    } else {
        queue_add(m->q, gettid());
        m->guard = 0;
        park();
    }
}

void unlock(lock_t *m) {
    while (TestAndSet(&m->guard, 1) == 1)
        ; //acquire guard lock by spinning
    if (queue_empty(m->q))
        m->flag = 0; // let go of lock; no one wants it
    else
        unpark(queue_remove(m->q)); // hold lock (for next thread!)
    m->guard = 0;
}
```

“Vaknar här”
Låset måste vara låst!



...det behövs nog någon typ av turordning!

```
typedef struct __lock_t {
    int flag;
    int guard;
    queue_t *q;
} lock_t;

void lock_init(lock_t *m) {
    m->flag = 0;
    m->guard = 0;
    queue_init(m->q);
}

void lock(lock_t *m) {
    while (TestAndSet(&m->guard, 1) == 1)
        ; //acquire guard lock by spinning
    if (m->flag == 0) {
        m->flag = 1; // lock is acquired
        m->guard = 0;
    } else {
        queue_add(m->q, gettid());
        m->guard = 0;
        park();
    }
}

void unlock(lock_t *m) {
    while (TestAndSet(&m->guard, 1) == 1)
        ; //acquire guard lock by spinning
    if (queue_empty(m->q))
        m->flag = 0; // let go of lock; no one wants it
    else
        unpark(queue_remove(m->q)); // hold lock (for next thread!)
    m->guard = 0;
}
```

setpark(); // new code



Linux

Lock!

mccccccc cccccccc cccccccc cccccccc

"Plats i kön!"

Wait if expected
else return...

Wake (in order)!

```
void mutex_lock (int *mutex) {
    int v;
    /* Bit 31 was clear, we got the mutex (this is the fastpath) */
    if (atomic_bit_test_set (mutex, 31) == 0)
        return;
    atomic_increment (mutex);
    while (1) {
        if (atomic_bit_test_set (mutex, 31) == 0) {
            atomic_decrement (mutex);
            return;
        }
        /* We have to wait now. First make sure the futex value
           we are monitoring is truly negative (i.e. locked). */
        v = *mutex;
        if (v >= 0)
            continue;
        futex_wait (mutex, v);
    }
}

void mutex_unlock (int *mutex) {
    /* Adding 0x80000000 to the counter results in 0 if and only if
       there are not other interested threads */
    if (atomic_add_zero (mutex, 0x80000000))
        return;

    /* There are other threads waiting for this mutex,
       wake one of them up. */
    futex_wake (mutex);
}
```



Ett konkret exempel, en räknare...

```
typedef struct __counter_t {
    int value;
} counter_t;

void init(counter_t *c) {
    c->value = 0;
}

void increment(counter_t *c) {
    c->value++;
}

void decrement(counter_t *c) {
    c->value--;
}

int get(counter_t *c) {
    return c->value;
}
```

Has critical sections => needs mutual exclusion => to be thread safe!



Ett konkret exempel, en räknare...

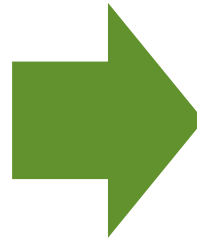
```
typedef struct __counter_t {
    int value;
} counter_t;

void init(counter_t *c) {
    c->value = 0;
}

void increment(counter_t *c) {
    c->value++;
}

void decrement(counter_t *c) {
    c->value--;
}

int get(counter_t *c) {
    return c->value;
}
```



```
typedef struct __counter_t {
    int value;
    pthread_mutex_t lock;
} counter_t;

void init(counter_t *c) {
    c->value = 0;
    Pthread_mutex_init(&c->lock, NULL);
}

void increment(counter_t *c) {
    Pthread_mutex_lock(&c->lock);
    c->value++;
    Pthread_mutex_unlock(&c->lock);
}

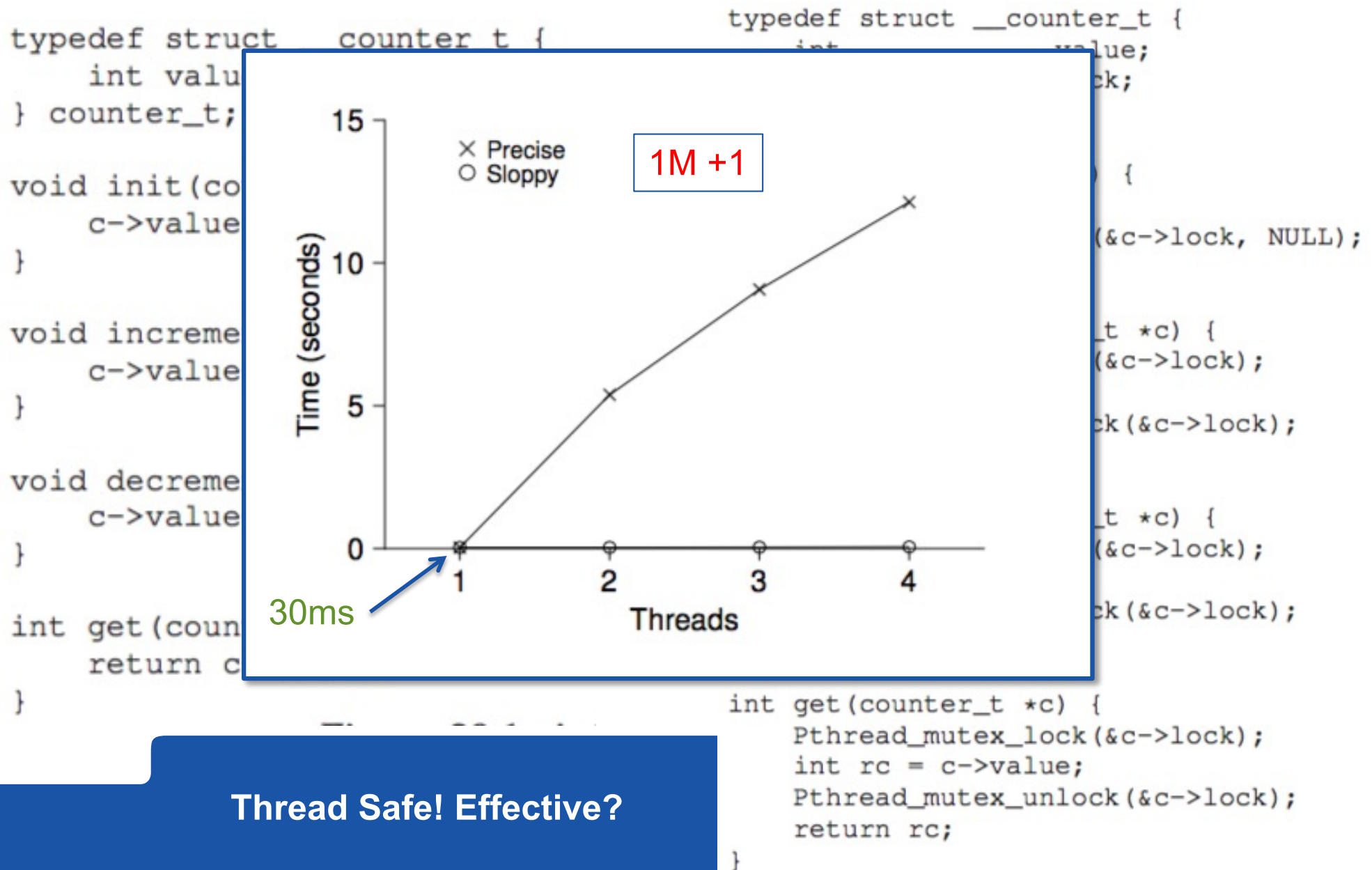
void decrement(counter_t *c) {
    Pthread_mutex_lock(&c->lock);
    c->value--;
    Pthread_mutex_unlock(&c->lock);
}

int get(counter_t *c) {
    Pthread_mutex_lock(&c->lock);
    int rc = c->value;
    Pthread_mutex_unlock(&c->lock);
    return rc;
}
```

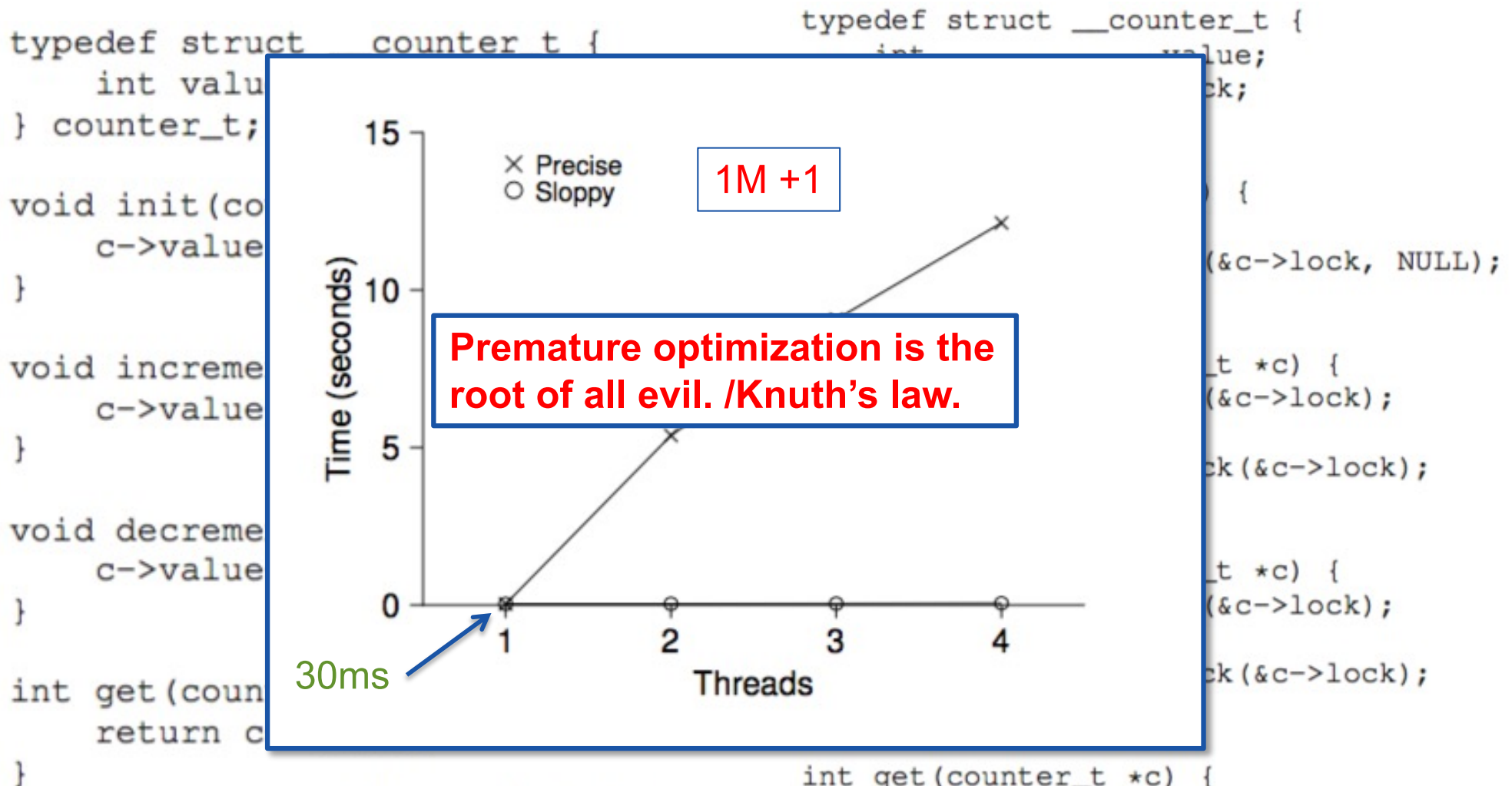
Thread Safe! Effective?



Ett konkret exempel, en räknare...



Ett konkret exempel, en räknare...



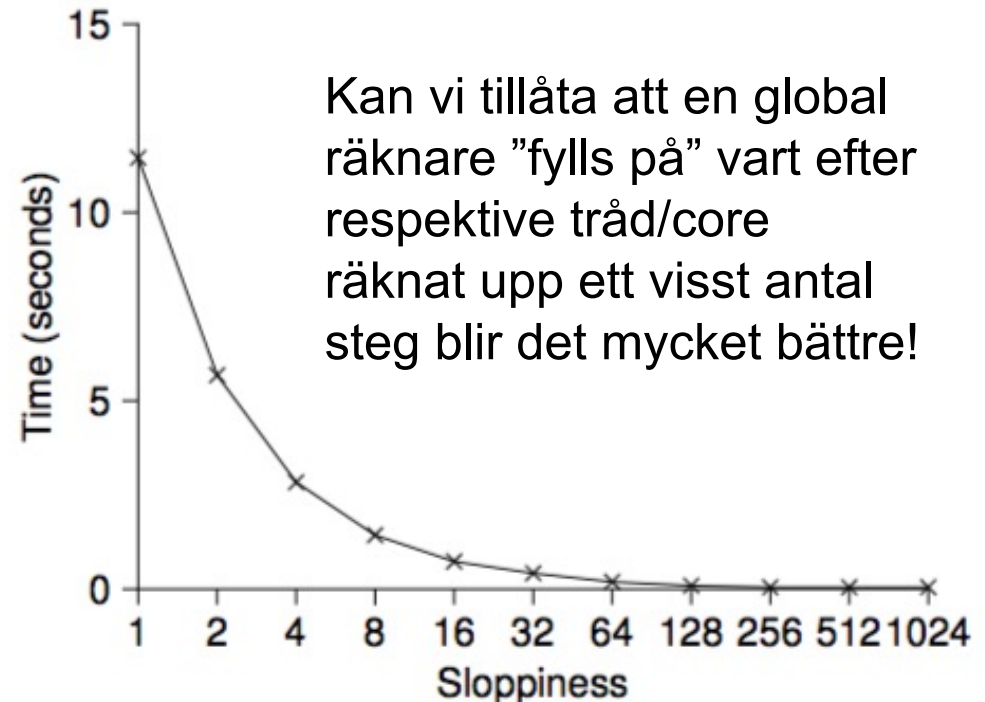
Big kernel lock (BKL)
If needed, optimize!

Ett konkret exempel, en räknare...

```
typedef struct __counter_t {  
    int          global;           // global count  
    pthread_mutex_t glock;         // global lock  
    int          local[NUMCPUS];   // local count (per cpu)  
    pthread_mutex_t llock[NUMCPUS]; // ... and locks  
    int          threshold;        // update frequency  
} counter_t;
```

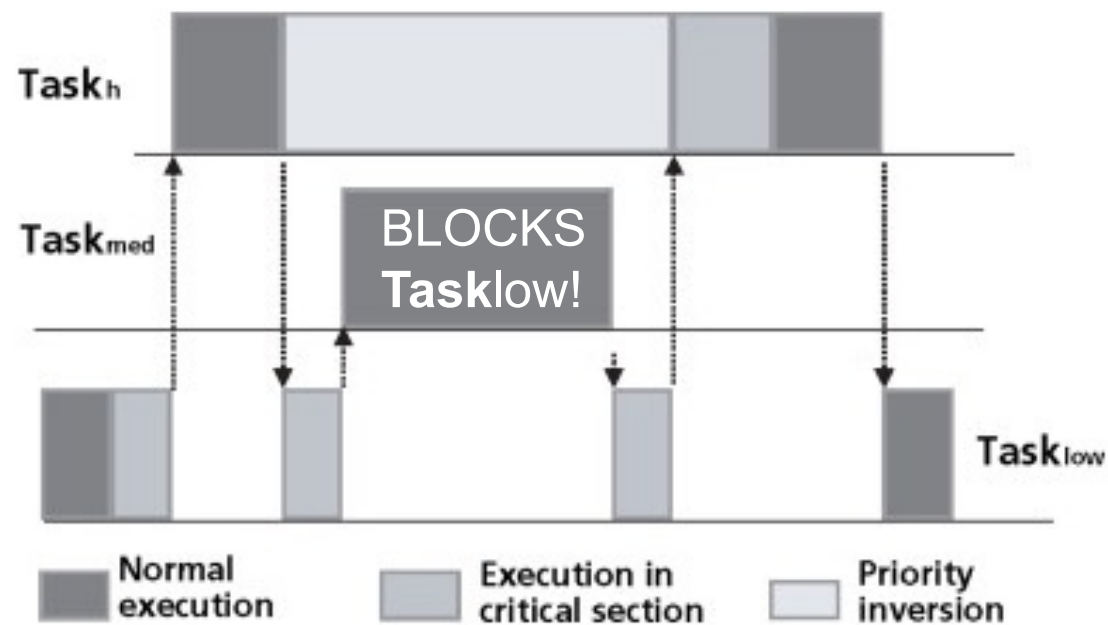
**”Sloppy
counter”**

Totalen blir alltid rätt,
men summan släpar lite!



Priority Inversion Problem

Står inte i boken,
ingår i kursen!





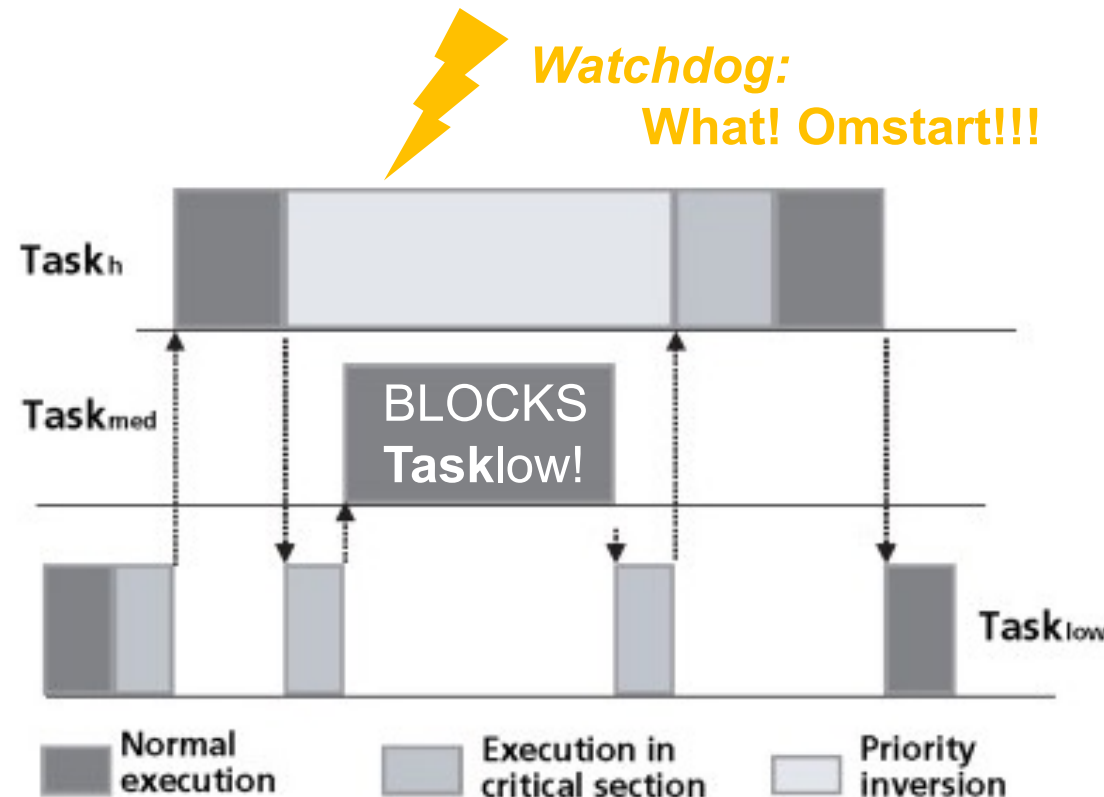
Priority Inversion Problem

Står inte i boken,
ingår i kursen!

Hantera "bussen", körs
ofta - under kort tid,
behöver då "buss lås".

Sänd bild till Jorden,
tar 15 minuter!!!

Lagra meteorologiska data
Sälla, kort tid, behöver
"buss lås" mycket kort tid.



Mars Pathfinder problem...

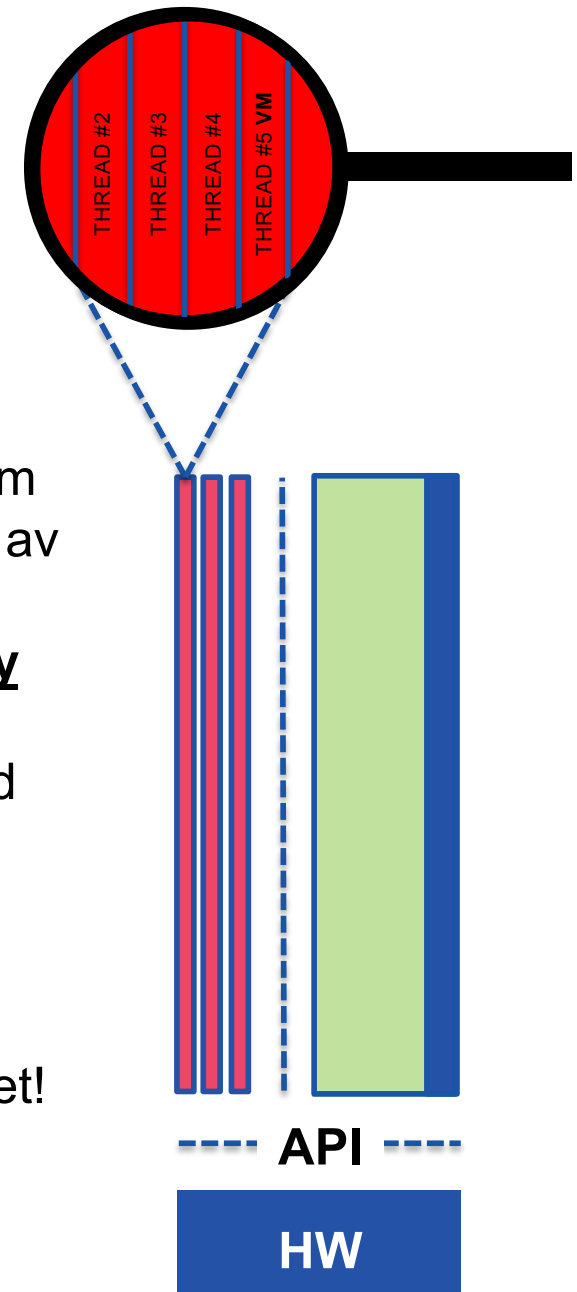
...solution: priority inheritance!

TASK_{low} får hög prioritet tills den lämnar ifrån sig låset!

En sista sak om trådar...

Det finns inget som hindrar att till exempel program som kompilerar till Byte code, och körs med hjälp av en virtuell maskin, implementerar trådar i den virtuella maskinen. Trådarna hanteras då av en ny schemaläggare i user space, och hela processen med alla sina trådar ser ut som en single threaded process ur os:ets synvinkel.

- + Det är lätt att byta/ställa in schemaläggaren
- + Trådbyte går blixsnabbt (ej trap till kernel mod)
- Om processen blir suspended blir alla trådar det!
- En process kan inte använda flera kärnor

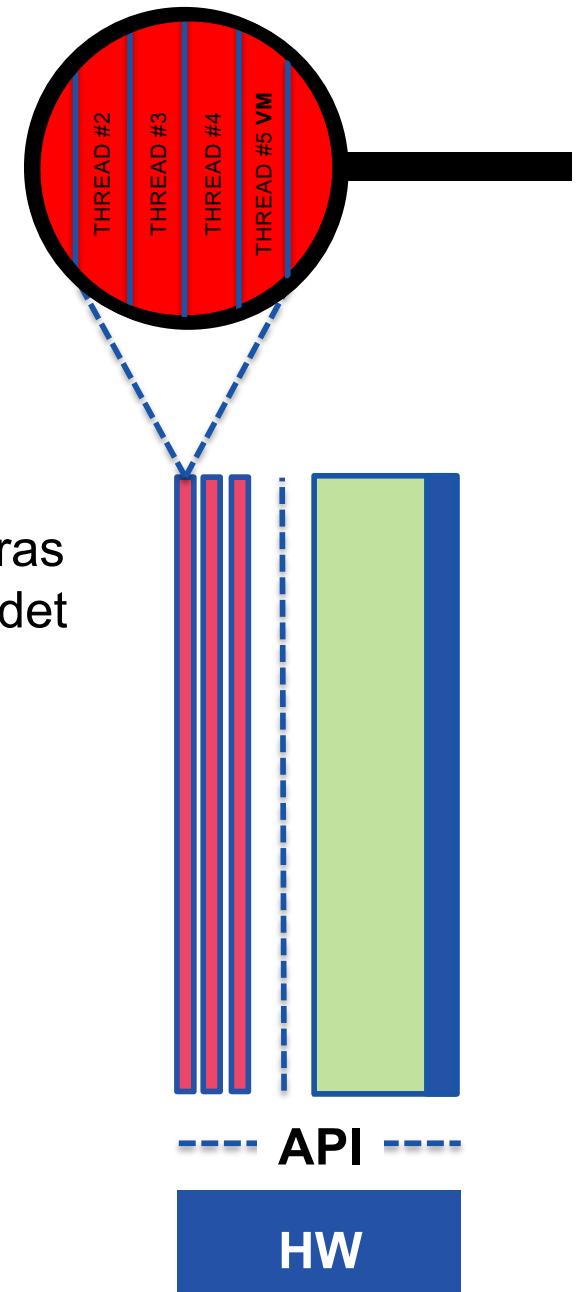


Java fungerade tidigare på detta sätt ("gröna" trådar) men inte längre...
...Haskell & Erlang har "gröna" trådar (men använder flera os trådar)!

En sista sak om trådar...

Omvänt så kan varje user space tråd implementeras med hjälp av en os tråd. Java fungerar numer på det sättet

- + En tråd kan bli suspended utan övriga trådar i processen blir suspended.
- + En process kan använda flera kärnor
 - Kräver trap till kernel mod
 - Os:ets schemaläggare gäller.





Begrepp

Inter Process Communication (IPC)

**Threads, POSIX API
Single vs. Multithreaded**

Race condition

Critical Section

Mutual exclusion

Atomic instruction

Test-and-set

Compare-and-swap/exchange

Load-link

Store-conditional

Fetch-and-add

Lock & Conditional Variables

Total Store Order, Priority Inversion

Spin lock, Sloppy Counter





Historik

1810xx AC 0.8 Skapad, saknas övning.

190110 AC 1.0 Release 1.0

2104xx AC 1.1 Mindre uppdateringar.